

DART-SD: Diamond-topology Aware Retrieval and Tuning for Self-Distillation of Multi-Turn Tool-Calling Agents

Hangrui Xu^{1,*}, Jiarui Wang^{1,*}, Yang Yang^{1,*†}, Chuanbo Zhu¹, Fangda Chen¹,
Ziqi Wu¹, Jingming Cai¹, Yan Song^{2,†}

¹ByteDance, ²University of Science and Technology of China

*Equal contribution, †Corresponding author

Abstract

Equipping Large Language Models (LLMs) with multi-turn tool-calling capabilities is essential for building autonomous agents. However, progress is fundamentally limited by the reliance on full-length trajectory imitation. For tasks involving multiple order-independent sub-goals, the optimal solution space forms a vast combinatorial diamond lattice. Forcing this rich topology into monolithic trajectories causes a severe *topological collapse*, indiscriminately penalizing valid alternative explorations and severely degrading policy diversity. To address this, we propose **DART-SD** (**D**iamond-topology **A**ware **R**etrieval and **T**uning for **S**elf-**D**istillation), a novel framework that shifts the paradigm from global forcing to topology-guided localized correction. DART-SD first models the execution process as a converging *Interaction-State Transition Graph (ISTG)*, faithfully capturing the inherent diamond topology of successful and failed exploratory paths. During autonomous rollouts, the framework identifies the *Critical Topological Breakpoint (CTB)* and retrieves success-supported recovery references. Finally, we introduce a *progressive self-distillation paradigm* through CTB-guided localized supervision, ensuring that the training loss is calculated exclusively on the generated recovery steps while strictly protecting the valid reasoning prefix from destructive gradient updates. Experiments on complex multi-turn tool-calling benchmarks demonstrate that DART-SD significantly outperforms traditional full-trajectory baselines.

Correspondence: Yang Yang at yang.yves@bytedance.com, Yan Song at clksong@gmail.com

1 Introduction

Equipping Large Language Models (LLMs) with multi-turn tool-calling capabilities is an essential milestone toward building autonomous agents capable of solving complex, real-world tasks [21, 30, 32]. Through sequential interactions with external environments, tool-augmented agents can retrieve up-to-date information, execute code, and manipulate APIs to transcend their static parametric knowledge [3, 4, 8, 31, 34]. Currently, state-of-the-art tool-calling performance is predominantly achieved by massive and expensive frontier models [1, 6]. Consequently, agent distillation has gained significant research momentum, aiming to transfer these sophisticated planning and interaction capabilities into efficient, compact open-source models [12, 34, 37].

The prevalent agent distillation paradigm relies on full trajectory imitation via Behavior Cloning (BC) or Supervised Fine-Tuning (SFT) [22]. In long-horizon tasks, global forcing paradigms apply an indiscriminate

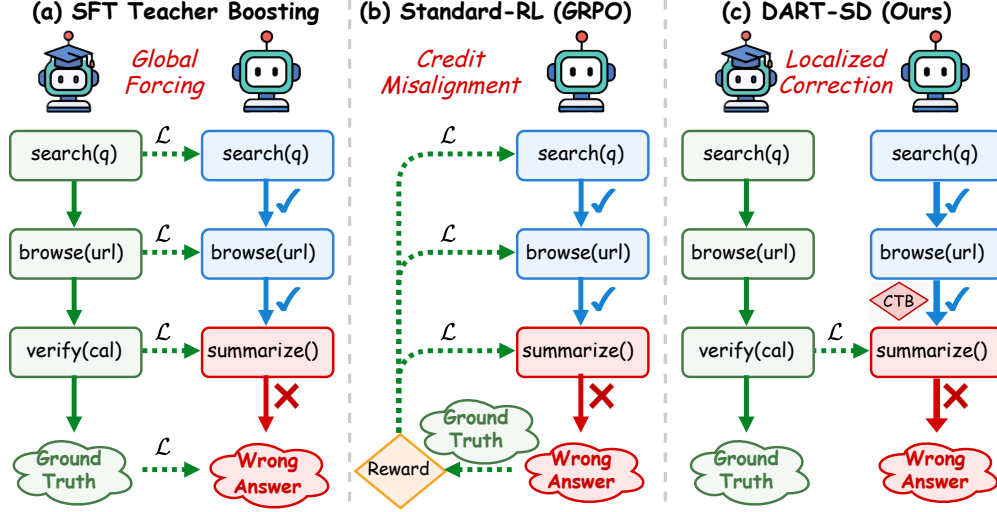


Figure 1 Comparison of training paradigms. (a) SFT Teacher Boosting applies an indiscriminate global loss, which overwrites valid exploration. (b) Standard-RL (GRPO) misassigns credit through a uniformly distributed reward spread. (c) DART-SD dynamically identifies the Critical Topological Breakpoints (CTB) and applies localized correction while preserving valid exploration.

global loss that overwrites the student model’s valid exploratory steps [17], causing models to memorize redundant trajectories instead of extracting the task’s core logical backbone. Recent efforts attempt to explore hindsight-based scaffolding or Reinforcement Learning (RL) [28, 39, 44]. However, traditional hindsight methods [40] still treat multi-turn interactions as strict linear sequences. This rigid linearity forces models to confuse fatal errors with harmless exploration, severely degrading token efficiency and internal knowledge consistency [18, 45]. Meanwhile, standard RL approaches [26, 29], such as Group Relative Policy Optimization (GRPO), frequently suffer from credit misassignment due to uniformly distributed rewards spread across all intermediate tool calls and inadvertently penalize valid steps within failed trajectories (see Figure 1).

The monolithic SFT and standard RL paradigms are fundamentally limited by misrepresenting the multi-turn tool-calling process as a collection of isolated linear trajectories [2, 11]. This oversimplified perspective ignores the inherent graph structure of state transitions [9, 13]. For tasks with order-independent sub-goals, distinct trajectories frequently intersect at shared intermediate states, structuring the optimal solution space into a vast combinatorial diamond lattice [7]. Forcing models to flatten this interconnected topological space into single linear paths, traditional methods inevitably cause a severe *topological collapse*. This structural oversight induces myopic credit assignment [29], arbitrarily penalizing valid alternative explorations and severely degrading policy diversity [15, 41].

To align with this topological reality, we propose **DART-SD** (**Diamond-topology Aware Retrieval and Tuning for Self-Distillation**), a framework that shifts the paradigm from linear imitation to topology-aware localized correction. DART-SD extracts a structural prior from teacher rollouts, instantiated as an *Interaction-State Transition Graph* (**ISTG**) covering both successful and failed trajectories. By defining nodes as cumulative interaction states rather than transient actions, the ISTG captures the diamond topology, naturally resolving order-dependency conflicts. During self-distillation, DART-SD projects student states onto the success-reachable region to identify the *Critical Topological Breakpoint* (**CTB**) and its recovery anchors. From each anchor, it searches the ISTG for a success-supported continuation as a reference for recovery generation. Finally, DART-SD applies a *progressive self-distillation paradigm* through CTB-guided localized supervision, ensuring that the loss is calculated only on recovery steps to protect valid reasoning prefixes.

In summary, our main contributions are as follows:

- We identify a fundamental topological collapse in existing agent-training paradigms. To resolve this, we introduce the Interaction-State Transition Graph (**ISTG**), which represents tool execution through cumu-

lative interaction states and captures the diamond topology induced by order-independent exploration.

- We define the *Critical Topological Breakpoint (CTB)* by projecting student interaction states onto the empirical success-reachable region. We propose *CTB-guided localized supervision*, which retrieves success-supported references and supervises the generated recovery steps while strictly preserving valid prefixes.
- We propose a *progressive self-distillation paradigm* that repeatedly rolls out the student, identifies its evolving capability boundary, and performs CTB-guided localized supervision. This iterative process enables the student to continuously extend valid interaction prefixes and progressively master more complex tool-use behaviors.
- Extensive experiments across five benchmarks and two model scales demonstrate that DART-SD consistently outperforms supervised and reinforcement-learning baselines, while reducing redundant tool calls and extending valid interaction prefixes.

2 Related Work

Distillation-Based Paradigms. Early attempts at building autonomous agents rely on Behavior Cloning (BC) or Supervised Fine-Tuning (SFT) over complete expert trajectories [22]. However, this global forcing paradigm forces students to mimic entire sequences, often leading to compounding errors in long-horizon tasks as deviations propagate autoregressively [16]. Furthermore, indiscriminate token-level imitation triggers exposure bias [25] and overwrites the student’s valid exploration habits. To address the limitations of static global forcing, recent distillation methods have shifted toward student-centered and dynamic strategies [14]. SCoRe-SFT [16] provides post-hoc scaffolding through student-centered knowledge distillation, while On-Policy Self-Distillation (OPSD) [44] mitigates offline distribution shifts by dynamically rolling out and distilling trajectories from the current student policy. Similarly, hindsight-based distillation methods such as HINT-SD [40] leverage targeted scaffolding for correction.

Reinforcement Learning Paradigms. To further refine policy optimization beyond distillation, Reinforcement Learning (RL) approaches leverage environment feedback to directly align agent behaviors. Methods such as FTRL-GRPO [39] and ToolRL [20] optimize agents using sparse outcome rewards and verifiable execution feedback. However, sparse terminal signals in these methods often lead to severe credit misassignment, inadvertently penalizing valid intermediate steps within failed trajectories [5, 17, 26, 27, 39, 43]. Even finer-grained RL methods such as MatchTIR [23], which aligns trajectories via bipartite matching to provide turn-level rewards, still fundamentally treat multi-turn interactions as rigid linear sequences, thereby overlooking the inherent topology of exploration. In contrast, our proposed DART-SD shifts the paradigm from linear imitation and complex matching to topology-aware localized correction. By calculating the optimization loss exclusively on retrieved recovery steps, DART-SD strictly protects valid reasoning prefixes from destructive gradient updates.

3 Methodology

Figure 2 provides an overview of the DART-SD framework. DART-SD first constructs an ISTG from teacher rollouts, then identifies the CTB of failed student rollouts through success-reachable projection. The identified CTB is used to generate localized recovery continuations and to drive progressive self-distillation.

3.1 ISTG Abstraction and Construction

Information Atom Abstraction. For each task x , useful facts extracted from tool responses are normalized into a task-specific set of information atoms $\mathcal{K}_x$. Semantically equivalent responses share the same atom, while non-informative responses contribute none. We realize this in two stages. A deterministic stage canonicalizes each tool response and decides whether it carries task-usable data at all. Parsing the response into fields, it is treated as non-informative only when every field is either a status signal or an empty or placeholder value, so a payload retaining a single substantive value still qualifies as informative, whereas an error or not-found

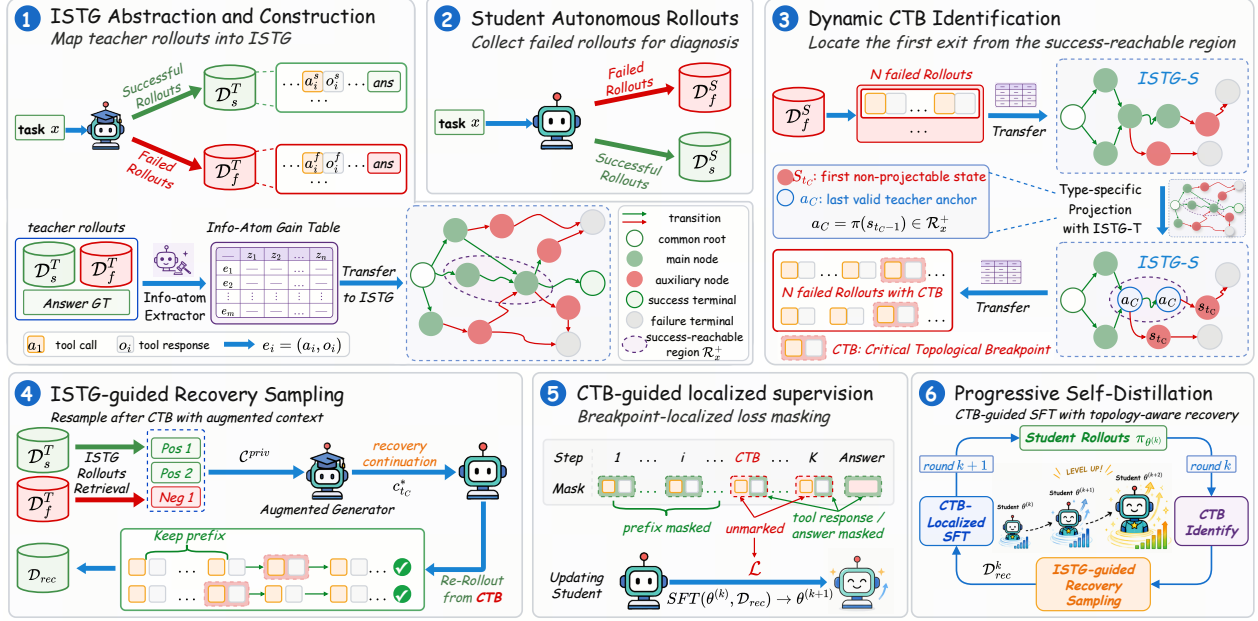


Figure 2 Overview of DART-SD. (1) DART-SD constructs an ISTG from teacher rollouts, where main and auxiliary nodes model information acquisition and useless exploration. (2–3) Failed student rollouts are collected and replayed in the same interaction-state space, then projected onto the budget-filtered success-reachable region $\mathcal{R}_x^+$, where the first projectable-to-non-projectable transition defines the CTB. (4–5) Conditioned on the retained student prefix and privileged teacher references, DART-SD generates a recovery continuation after the CTB and applies localized supervision only to the generated assistant tokens. (6) This CTB-guided SFT loop is progressively repeated across self-distillation rounds.

message does not. Non-informative responses are collapsed per tool into a single class, which removes the bulk of the corpus without semantic judgement.

A semantic stage then assigns atoms to the surviving candidates. Let e denote a tool call, with tool identity $\text{tl}(e)$ and canonicalized response $\bar{o}(e)$. All candidates of a task are examined jointly, conditioned on the task question and one successful rollout, yielding a set-valued atom map

$$\alpha_x : (\text{tl}(e), \bar{o}(e)) \mapsto \alpha_x(e) \subseteq \mathcal{K}_x, \quad |\alpha_x(e)| \leq 1. \quad (1)$$

An informative response is mapped to the singleton containing its atom, while a non-informative response, or one left unjudged, is mapped to $\emptyset$. Responses supplying the same fact are constrained to receive the same atom, so each element of $\mathcal{K}_x$ is the canonical representative of one class of information-equivalent calls, namely those whose images under α_x coincide and are non-empty. Judging a task’s candidates jointly is what makes this equivalence decidable: atoms are assigned with all alternatives in view, rather than labeling responses independently and relying on the labels to agree.

The map α_x supplies the acquisition increment used in Eq. (4). Writing B_t for the tool call or concurrent call bundle executed at step t ,

$$\Delta I_t = \{k \in \mathcal{K}_x \mid \exists e \in B_t, k \in \alpha_x(e), k \notin I_{t-1}\}, \quad I_t = I_{t-1} \cup \Delta I_t. \quad (2)$$

Two properties of this construction determine the resulting topology. First, α_x identifies responses across surface form and across tools, so the same fact returned as free text by one tool and as a structured record by another yields one atom; the two acquisition paths therefore reconverge at the same main information state instead of forking into states the projection would treat as unrelated. Second, mapping non-informative and unjudged responses to $\emptyset$ is what makes them contribute none: such a call contributes no element to ΔI_t in Eq. (2) and leaves the information set unchanged, instead of receiving an identifier derived from its

own text, which would manufacture a distinct information state that no evidence supports and inflate the information dimension of the graph. Because only informative responses yield atoms, residual judgement errors predominantly omit an atom rather than invent one, leaving the graph sparser but never fabricating states or reachability.

Main and Auxiliary Nodes. For a task x , the interaction state at step t is represented as

$$X_t = (I_t, U_t), \quad (3)$$

where I_t is the set of canonical information atoms acquired up to step t , and U_t records the multiset of useless operations performed since the most recent main node.

The root state $X_0 = (\emptyset, \emptyset)$ is defined as a main node. For $t \geq 1$, the type of X_t is determined by the preceding transition:

$$\text{type}(X_t) = \begin{cases} \text{main}, & \Delta I_t \neq \emptyset, \\ \text{aux}, & \Delta I_t = \emptyset \text{ and } U_t \neq \emptyset. \end{cases} \quad (4)$$

where ΔI_t is the set of previously unseen information atoms acquired at step t , given by Eq. (2).

Main nodes include mixed steps acquiring new information alongside useless operations and are represented with $U_t = \emptyset$, whereas auxiliary nodes record useless operations after the main node without changing the information set.

Interaction-State Transition Graph (ISTG). For each task x , we construct a directed multigraph

$$G_x = (V_x, E_x). \quad (5)$$

The graph is built from all successful and failed teacher rollouts. All rollouts share a common root and terminate at either a success or a failure terminal. Each vertex $v \in V_x$ represents a state $X(v) = (I(v), U(v))$ with type $\text{type}(v) \in \{\text{main}, \text{aux}\}$, and each edge represents one tool call or concurrent call bundle. Parallel edges are retained.

Main nodes form the information-acquisition backbone, while auxiliary nodes represent useless exploration attached to the most recent main node. Both node types and transitions are retained, since auxiliary exploration contributes to execution depth and matching. The set-valued information component allows order-independent acquisition paths to diverge and reconverge at the same main information state, producing the diamond structures of the ISTG. A student rollout is replayed under the same state-update rule:

$$P_x^s = (s_0, \dots, s_T), \quad X(s_t) = (I_t^s, U_t^s). \quad (6)$$

The node type of s_t is determined by the preceding transition. Teacher and student executions are therefore embedded in a shared interaction-state space, enabling separate matching of information acquisition and useless exploration without requiring action-wise correspondence.

3.2 Success-Reachable Projection and CTB

Empirical Success-Reachable Region. Let $\mathcal{T}_x^+$ denote the successful teacher rollouts for task x , and let $d_x^{\min}$ be the minimum number of graph transitions from the root to a success terminal among these rollouts. We define the task-specific reachability budget as

$$B_x = \min(d_x^{\min} + \Delta_x, B_x^{\max}), \quad (7)$$

where Δ_x is the task-dependent allowance beyond the shortest successful depth and $B_x^{\max}$ is the budget cap. For a node v on a successful rollout τ , let $r_\tau(v)$ denote the number of graph transitions remaining from v to the success terminal of τ . The empirical success-reachable region is defined by

$$\mathcal{R}_x^+ = \{v \in V_x \mid \exists \tau \in \mathcal{T}_x^+, v \in \tau, r_\tau(v) \leq B_x\}. \quad (8)$$

Thus, $\mathcal{R}_x^+$ contains both main and auxiliary nodes on successful teacher rollouts whose remaining distance to success is at most B_x . The distance is measured backward from the success terminal along the rollout, rather than as forward depth from the root. States observed exclusively on failed rollouts remain in G_x , but are excluded from $\mathcal{R}_x^+$ and cannot serve as recovery anchors.

Type-specific State Projection. For each student state $s_t = (I_t^s, U_t^s)$, we further apply the projection rule associated with its node type. Let $\mathcal{R}_{x,\text{main}}^+$ and $\mathcal{R}_{x,\text{aux}}^+$ denote the main and auxiliary nodes in the budget-filtered empirical success-reachable region.

For a student main node, the acceptable teacher anchors are defined over non-root successful nodes with valid information acquisition:

$$\mathcal{A}_t^{\text{main}} = \left\{ v \in \mathcal{R}_{x,\text{main}}^+ \mid I(v) \subseteq I_t^s \right\}. \quad (9)$$

Thus, a student main node can be aligned with any reachable teacher main node whose information set is contained in the student's acquired information. Equality of information sets and the useless-operation component are not required. For mixed transitions, the criterion depends only on the information component, since any transition with information gain is classified as a main transition.

For a student auxiliary node, we first consider the reachable teacher main nodes whose information is contained in I_t^s :

$$\mathcal{M}_t = \left\{ m \in \mathcal{R}_{x,\text{main}}^+ \mid I(m) \subseteq I_t^s \right\}. \quad (10)$$

If $\mathcal{M}_t \neq \emptyset$, let m_t be a node in $\mathcal{M}_t$ with the largest information-set cardinality. We then define the acceptable auxiliary anchors as:

$$\mathcal{A}_t^{\text{aux}} = \left\{ v \in \mathcal{R}_{x,\text{aux}}^+ \mid \text{par}(v) = m_t, |U(v)| = |U_t^s| \right\}, \quad (11)$$

where $\text{par}(v)$ denotes the reachable teacher main node from which the auxiliary node v is reached. Thus, the projection compares only the number of useless operations performed after m_t , without requiring the unsuccessful tools to be identical. If $\mathcal{M}_t = \emptyset$, then $\mathcal{A}_t^{\text{aux}} = \emptyset$. Let $\mathcal{A}_t$ be $\mathcal{A}_t^{\text{main}}$ or $\mathcal{A}_t^{\text{aux}}$ according to the type of s_t , and define

$$\rho_t = \mathbb{I}[\mathcal{A}_t \neq \emptyset]. \quad (12)$$

If $\rho_t = 1$, the student state corresponds to at least one reachable teacher node of the same type. We formally denote the selected valid teacher anchor as $\pi(s_t)$: for a main node, $\pi(s_t)$ is chosen from $\mathcal{A}_t^{\text{main}}$ as a node with the maximal information set; for an auxiliary node, $\pi(s_t)$ can be any anchor in $\mathcal{A}_t^{\text{aux}}$. If $\rho_t = 0$, no valid same-type anchor exists and the student state is treated as outside the success-reachable region.

Critical Topological Breakpoint (CTB). Since the initial state is shared by the teacher and student, $\rho_0 = 1$. The CTB is defined as the first transition from a projectable to a non-projectable student state:

$$\begin{aligned} t_C &= \min\{t : \rho_{t-1} = 1, \rho_t = 0\}, \\ a_C &= \pi(s_{t_C-1}), \end{aligned} \quad (13)$$

where $\pi(\cdot)$ denotes the projection mapping that assigns a valid student node to its teacher anchor. Thus, t_C identifies the first departure from the teacher-supported region, and the anchor a_C is the last valid teacher projection before that departure. This boundary captures the earliest point at which the student either acquires an unsupported information combination or exceeds the teacher-supported exploration count under the corresponding main node. If the student remains projectable until the end of the rollout but still fails, we instead view the terminal state as the correction boundary and use its latest valid projection as the recovery anchor.

3.3 CTB-Guided Localized Supervision

Privileged-context Retrieval. We randomly sample successful and failed teacher traces from the teacher graph as privileged references, denoted as $\mathcal{C}^{\text{Priv}}$. The reference traces are not directly concatenated with the student prefix, since the two trajectories may contain different calls and observations. Instead, an augmented generator is conditioned on the task, the student prefix retained before the CTB, and the privileged references:

$$c_{t_C}^* = \text{AugGen}(x, \tau_{<t_C}^s, \mathcal{C}^{\text{priv}}). \quad (14)$$

The training trajectory is formed by appending $c_{t_C}^*$ to the retained student prefix. The trajectory is then tokenized, and CTB-localized supervision is applied to the resulting training sequence. Thus, the realized student context is preserved, while the sampled teacher traces provide privileged information for generating the subsequent continuation.

CTB-Localized Supervision. Let $\tilde{y} = (\tilde{y}_1, \dots, \tilde{y}_L)$ denote the tokenized training trajectory. We train with the masked causal language modeling objective:

$$\mathcal{L}_{\text{DART}} = - \sum_{i=1}^L m_i \log p_{\theta}(\tilde{y}_i \mid \tilde{y}_{<i}, x). \quad (15)$$

Although the loss is computed at the token level, the mask is defined over response steps: $m_i = 1$ only if token $\tilde{y}_i$ belongs to an assistant response step generated after the CTB and before the final-answer step. Tokens in the retained student prefix, user messages, tool observations, and the final-answer step receive zero weight. Thus, supervision is restricted to post-CTB assistant response steps, while the pre-CTB behavior and final answer are not directly optimized.

3.4 Progressive Self-Distillation Paradigm

The task-specific ISTG is maintained throughout the self-distillation. At each iteration, the current student produces new rollouts, which are mapped into the shared interaction-state space and processed by the type-specific projection and breakpoint-localization procedure. Failed rollouts yield breakpoint-localized privileged-context distillation instances. As the student improves, its states remain projectable for longer portions of the interaction trajectory. The detected CTBs therefore track the evolving boundary at which the current policy first departs from the teacher-supported region. DART-SD consequently induces a self-paced curriculum that progressively shifts supervision toward recovery behaviors beyond the current tool-use capability.

4 Experiments

4.1 Experimental Setup

Datasets. We train the models on the **FTRL** dataset [39], which comprises over 2,000 automatically constructed tool-use environments with verifiable feedback. FTRL covers four types of task structures: Single, containing a single sub-question; Para-Single, consisting of multiple independent sub-questions executable in parallel; Multi, comprising a sequence of dependent sub-questions; and Para-Multi, combining both independent and dependent sub-questions. We evaluate DART-SD on five tool-use benchmarks, including FTRL as the in-domain test set, and **BFCL** [19], **ToolHop** [38], τ -**bench** [35], and **RoTBench** [36] as out-of-domain benchmarks to assess generalization.

Baselines. We evaluate DART-SD using both Qwen3-4B and Qwen3-8B backbones [33]. For each backbone, we first report the performance of the pretrained model without task-specific adaptation, denoted as *Base*. The remaining baselines are grouped into two optimization paradigms. **(1) Distillation-based Paradigms:** standard *SFT*, *SCoRe-SFT* [16], and *OPSD* [44]. **(2) RL Paradigms:** *FTRL-GRPO* [39], *ToolRL* [20], and *MatchTIR* [23] (evaluated using both *MatchTIR-OT* and *MatchTIR-KM*). To ensure a strictly controlled comparison, all methods are trained and evaluated under a no-thinking configuration.

Evaluation Metrics. To ensure a rigorous and standardized assessment, we strictly adhere to the official evaluation protocols for each respective benchmark. **(1) FTRL:** We evaluate trajectory precision and task completeness using *Solve-P* and *Solve-R* along with their harmonic mean, *Solve-F1*. **(2) BFCL:** We report the average *Multi-Turn* score over Base, Miss Function, Miss Parameter and Long Context. **(3) ToolHop:** We evaluate the model’s ability to resolve complex multi-hop tool dependencies using *Answer Correctness (AC)*. **(4) τ -bench:** We assess performance on real-world API orchestration tasks using *Pass[^]1*. **(5) RoTBench:** We

Methods	FTRL			BFCL	ToolHop	τ -bench	RoTBench			Avg.
	Solve-P	Solve-R	Solve-F1	Multi-Turn	AC	Pass^1	TS	PI	CF	
Qwen3-4B										
Base	21.00	26.54	21.81	10.14	20.20	15.15	69.52	26.31	16.07	25.19
♠ SFT	34.26	49.02	37.96	14.57	40.50	21.82	71.67	44.40	24.40	37.62
♠ SCoRe-SFT	23.79	28.20	24.61	12.75	25.83	11.52	60.36	31.19	18.33	26.29
♠ OPSD	22.37	29.21	23.50	11.00	20.20	23.03	68.93	35.24	21.67	28.35
◇ FTRL-GRPO	36.83	41.71	37.84	13.50	29.25	20.61	70.36	32.62	20.24	33.66
◇ ToolRL	26.89	33.23	28.47	9.88	20.60	18.18	69.64	38.21	24.05	29.91
◇ MatchTIR (OT)	23.55	29.27	24.95	10.38	26.23	16.36	72.02	32.74	20.36	28.43
◇ MatchTIR (KM)	25.54	31.10	26.50	10.00	26.63	21.82	72.02	33.81	21.43	29.87
♠ DART-SD (Ours)	36.70	48.16	39.77	23.88	42.11	23.03	72.02	42.38	24.52	39.17
Qwen3-8B										
Base	21.18	30.71	23.48	18.38	28.54	10.13	75.52	36.29	22.19	29.60
♠ SFT	38.08	50.95	41.89	19.25	43.52	26.06	75.95	48.81	30.24	41.64
♠ SCoRe-SFT	31.28	34.48	31.58	19.25	30.25	18.18	70.36	40.48	24.52	33.38
♠ OPSD	24.37	34.88	26.68	20.50	41.11	21.21	75.24	43.21	27.38	34.95
◇ FTRL-GRPO	37.66	45.49	40.22	35.25	34.57	23.03	77.02	42.74	27.02	40.33
◇ ToolRL	32.49	41.07	35.00	20.50	44.72	25.45	75.71	51.43	33.10	39.94
◇ MatchTIR (OT)	29.45	34.89	30.65	21.38	38.79	24.24	75.00	40.36	25.36	35.57
◇ MatchTIR (KM)	33.07	40.42	35.37	23.25	41.71	26.06	75.71	42.26	26.90	38.31
♠ DART-SD (Ours)	42.00	54.13	45.66	27.63	45.03	27.12	75.83	57.38	35.48	45.58

Table 1 Performance comparison of different training methods on five tool-use benchmarks using Qwen3-4B and Qwen3-8B backbones. Training-based baselines are organized into distillation-based ♠ and reinforcement learning ◇ paradigms. All trainable methods are trained on FTRL and evaluated on both the in-domain FTRL test set and four out-of-domain benchmarks. The **best** and **second-best** results within each backbone are highlighted.

evaluate fine-grained execution robustness through *Tool Selection (TS)*, *Parameter Identification (PI)*, and *Content Filling (CF)*.

Implementation Details. We initialize the student model from Qwen3-4B and Qwen3-8B. Teacher trajectories are collected from a mixed pool of Qwen3.6-27B [24] and GLM-5.2 [42]. We perform five iterations of CTB-guided localized SFT over all 2,215 tasks in the FTRL training set. At each iteration, the current student checkpoint generates eight trajectories per task with a temperature of 0.7, a maximum generation length of 4,096 tokens, and at most nine interaction turns. The model is then fine-tuned for one epoch using breakpoint-localized loss masking, with a learning rate of 5×10^{-7} and a batch size of 32. Each augmented training context contains two positive references and one negative reference, and each positive reference is accompanied by a teacher-generated analysis.

4.2 Experimental Results

Overall Performance. As shown in Table 1, DART-SD demonstrates robust and superior performance on both the in-domain FTRL dataset and four out-of-domain tool-use benchmarks. Compared with distillation-based methods and reinforcement learning methods, DART-SD delivers consistent empirical gains and achieves the strongest overall performance. This robust cross-domain superiority confirms that our topology-aware localized tuning learns generalizable and transferable tool-use capabilities, rather than merely overfitting to the FTRL training distribution. Crucially, DART-SD maintains strong performance across different parameter scales. It achieves the highest average performance with both the Qwen3-8B and Qwen3-4B backbones, while obtaining the best or competitive results on most individual metrics. These results indicate that the effectiveness of topology-aware localized tuning is not limited to a particular model size and remains robust when applied to a smaller backbone.

Effectiveness of Capability Distillation. Figure 3 compares the performance of the base Qwen3-8B student, DART-SD, and the teacher across five tool-use benchmarks. For visualization, we report Solve-F1 on FTRL, the average Multi-Turn score on BFCL, AC on ToolHop, Pass^1 on τ -bench, and PI on RoTBench. Initialized from the same 8B base model, DART-SD delivers substantial improvements across all five benchmarks. Notably,

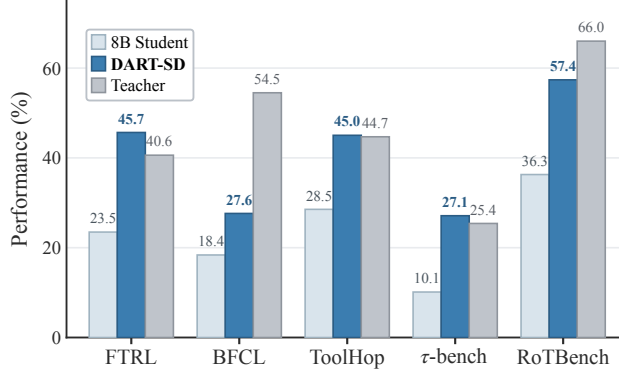


Figure 3 Performance comparison of Qwen3-8B, DART-SD, and the teacher across five tool-use benchmarks. DART-SD improves upon Qwen3-8B on all benchmarks and **surpasses** the teacher on FTRL, ToolHop, and τ -bench.

the resulting student surpasses the teacher on FTRL, ToolHop, and τ -bench, despite using substantially fewer parameters. These results suggest that DART-SD goes beyond direct trajectory imitation. By leveraging topology-aware recovery paths, it effectively extracts and generalizes reusable tool-use behaviors from the structural knowledge encoded in the ISTG, enabling the smaller student to outperform its teacher on several benchmarks.

Task	DART-SD					Golden
	Iter1	Iter2	Iter3	Iter4	Iter5	
Single	1.23	1.15	1.10	1.09	1.07	1.00
Multi	3.83	4.63	4.54	4.49	4.41	4.71
Para-Single	3.29	2.65	2.27	2.26	2.22	2.11
Para-Multi	7.25	6.48	5.97	5.82	5.62	6.97
Overall	4.23	3.99	3.71	3.65	3.55	4.02
Solve-F1	40.37	42.95	43.78	44.67	45.66	—

Table 2 Average tool-call length of successful trajectories across progressive SFT iterations on the FTRL test set. DART-SD progressively improves Solve-F1 while shortening its tool traces, eventually producing more efficient traces than the golden solutions given during data construction.

Tool-Call Length of Successful Trajectories. To verify whether DART-SD improves reasoning rather than memorizing references, we track the evolution of successful trajectories across iterations. We measure task performance (*Solve-F1*) and efficiency (average tool calls) on the FTRL test set, and compare the final model against the golden references. The golden reference denotes the tool trajectory provided by the FTRL construction pipeline. As shown in Table 2, *Solve-F1* steadily improves while trajectory length decreases. The average number of tool calls drops from 4.23 at Iter1 to 3.55 at Iter5. This confirms that the model learns **more efficient tool-use strategies** instead of relying on brute-force exploration or longer execution chains. This reduction is especially clear in complex multi-step and parallel tasks, suggesting that our method effectively removes redundant tool calls while maintaining high success rates. Remarkably, the trajectories of our final model are even **shorter than the golden references** (3.55 vs. 4.02). This highlights that DART-SD successfully discovers optimized shortcuts rather than blindly following the provided subtask structures.

CTB Position Shifts within Failed Trajectories. To analyze how the student’s capability boundary expands during progressive training, we track the average CTB position within failed trajectories across iterations. A later CTB position indicates that the model remains within the empirically recoverable region for a longer prefix before its first departure. As shown in Table 3, the average CTB position steadily advances from 0.348 at Iter1 to 1.452 at Iter5, indicating that the first departure from recoverable behavior is progressively delayed. This trend is particularly evident in complex multi-hop and parallel multi-hop tasks, where increasingly longer valid prefixes are learned. These results validate the effectiveness of CTB-guided localized supervision. By

Task	CTB Position				
	Iter1	Iter2	Iter3	Iter4	Iter5
Single	0.034	0.186	0.137	0.249	0.077
Multi	0.500	1.624	1.729	1.876	1.953
Para-Single	0.095	0.219	0.299	0.328	0.313
Para-Multi	0.395	1.524	1.692	1.791	1.816
Overall	0.348	1.185	1.310	1.421	1.452
Δ vs. Iter1	–	+0.837	+0.962	+1.073	+1.104

Table 3 Average CTB positions of failed training trajectories across progressive SFT. Larger values indicate that the first departure from empirically recoverable behavior occurs later, meaning that the model correctly executes a longer trajectory prefix before localized recovery is required.

identifying the current capability boundary, DART-SD performs teacher-guided resampling from the first departure point and supervises only the recovery suffix while preserving already-mastered prefixes. As the boundary moves deeper across iterations, the student progressively acquires more complex tool-use behaviors, demonstrating the effectiveness of DART-SD for long-horizon tasks.

Method	FTRL	BFCL	ToolHop
Qwen3-8B	29.74	40.00	42.21
FTRL-GRPO	32.85	41.50	36.72
ToolRL	26.72	34.25	32.93
MatchTIR (KM)	<u>37.33</u>	<u>47.13</u>	<u>46.16</u>
DART-SD	41.03	49.75	46.43

Table 4 Performance comparison under the thinking setting. Results are reported using FTRL Solve-F1, the average BFCL Multi-Turn score, and ToolHop AC.

Effect of Thinking Mode. Table 4 evaluates DART-SD under the thinking setting. All trainable baselines are trained with thinking enabled, whereas the Qwen3-8B base model enables thinking only at inference time. For DART-SD, teacher trajectories are generated in thinking mode, and both the teacher’s reasoning traces and tool-call trajectories are incorporated into the resampling context for student training. The effect of explicit thinking varies across methods and benchmarks. To provide a fair and controlled comparison, we therefore use the no-thinking configuration in all other experiments unless otherwise specified, allowing us to more directly evaluate the proposed training framework. Importantly, DART-SD still achieves the best performance across all three benchmarks under the thinking setting, demonstrating that CTB-guided localized supervision remains effective when explicit thinking is enabled.

Method	IFEval	AIME24	AIME25	MMLU	Avg.
Qwen3-8B	34.75	46.67	23.33	70.94	43.92
SFT	35.30	43.33	26.67	71.43	44.18
DART-SD	45.29	50.00	30.00	74.27	49.89

Table 5 General capability evaluation on representative benchmarks. The **best** results are highlighted.

General Capability Preservation. To evaluate whether DART-SD preserves the model’s general capabilities beyond tool-use tasks, we further evaluate it under a thinking-enabled setting on four standard benchmarks: IFEval [46] for instruction following, AIME24 and AIME25 for mathematical reasoning, and MMLU [10] for general knowledge. We report strict prompt-level accuracy on IFEval, pass@10 on AIME24 and AIME25, and accuracy on MMLU. As shown in Table 5, DART-SD consistently outperforms both the pretrained model and standard SFT across all benchmarks, improving the average score from 43.92 to 49.89. These results suggest that topology-aware localized supervision not only enhances tool-use ability but also preserves the model’s general reasoning and instruction-following capabilities.

Method	Solve-P	Solve-R	Solve-F1
Qwen3-8B	21.18	30.71	23.48
+SD	36.23	44.46	38.10
+CTB	36.62	46.32	39.51
+Progressive SFT	41.32	49.65	43.93
+ISTG (Ours)	42.00	54.13	45.66

Table 6 Component ablation of DART-SD on the FTRL test set using Solve-P, Solve-R, and Solve-F1.

Ablation Study on Different Components. Table 6 presents the component ablation of DART-SD to evaluate the contribution of each core design. First, applying Self-Distillation (SD) directly over the base model yields substantial gains, demonstrating the efficacy of learning from the agent’s own exploratory trajectories. Second, introducing CTB-guided localized supervision further improves performance by restricting the loss calculation exclusively to the post-breakpoint recovery suffix, thereby avoiding direct global supervision on already-mastered prefixes. Third, Progressive Self-Distillation introduces iterative rollout and localized correction rounds, confirming the necessity of a continuous iterative adaptation paradigm over one-shot training. Finally, ISTG replaces LLM-judge-based breakpoint detection with topology-aware state projection and structured recovery, enabling more accurate CTB identification and structurally compatible supervision. Together, these complementary components yield the strongest performance.

5 Conclusion

In this paper, we propose **DART-SD**, a topology-aware self-distillation framework for multi-turn tool-calling agents. DART-SD models execution as an *Interaction-State Transition Graph (ISTG)*, revealing the diamond topology induced by order-independent valid exploration. Building on the ISTG, DART-SD identifies the *Critical Topological Breakpoint (CTB)*, retrieves recovery references from the ISTG, and performs progressive self-distillation via CTB-guided localized supervision while preserving the valid interaction prefix. Extensive experiments demonstrate that DART-SD consistently outperforms both distillation and reinforcement learning baselines across multiple in-domain and out-of-domain tool-use benchmarks. Beyond improving task success, DART-SD enables more efficient tool-use behaviors by preserving exploration, reducing redundant tool calls, and progressively mastering more complex tool-use behaviors throughout self-distillation. More broadly, our findings suggest that effective agent distillation should be guided by interaction-state topology rather than rigid trajectory imitation. We hope this perspective encourages future research on structure-aware agent training and graph representations for long-horizon reasoning and decision-making.

References

- [1] Anthropic. Introducing claude 4, 2025. URL <https://www.anthropic.com/news/claude-4>.
- [2] Huacan Chai, Zijie Cao, Maolin Ran, Yingxuan Yang, Jianghao Lin, Xin Peng, Hairui Wang, Renjie Ding, Ziyu Wan, Muning Wen, et al. Parl-*mt*: Learning to call functions in multi-turn conversation with progress awareness. *arXiv preprint arXiv:2509.23206*, 2025.
- [3] Zhuohong Chen, Zhengxian Wu, Zirui Liao, Shenao Jiang, Hangrui Xu, Yang Chen, Chaokui Su, Xiaoyu Liu, and Haoqian Wang. R3g: A reasoning-retrieval-reranking framework for vision-centric answer generation. In *ICASSP 2026 - 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 8602–8606, 2026. doi: 10.1109/ICASSP55912.2026.11463453.
- [4] Zhuohong Chen, Zhenxian Wu, Yunyao Yu, Hangrui Xu, Zirui Liao, Zhifang Liu, Xiangwen Deng, Pen Jiao, and Haoqian Wang. Learning to search: A decision-based agent for knowledge-based visual question answering. *arXiv preprint arXiv:2604.07146*, 2026.
- [5] Xin Cheng, Shuo He, Lang Feng, HaiYang Xu, Ming Yan, Lei Feng, and Bo An. Beyond trajectory-level attribution: Graph-based credit assignment for agentic reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2026.
- [6] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- [7] Joan Vendrell Gallart, Solmaz Kia, Russell Bent, and Michael Grosskopf. Chain-based adaptive reconfiguration over lattices for hallucination reduction. *arXiv preprint arXiv:2605.27706*, 2026.
- [8] Zorik Gekhman, Roei Aharoni, Eran Ofek, Mor Geva, Roi Reichart, and Jonathan Herzig. Thinking to recall: How reasoning unlocks parametric knowledge in llms. *arXiv preprint arXiv:2603.09906*, 2026.
- [9] Lakshya Gupta, Litao Li, Yizhe Liu, Sriram Ganapathi Subramanian, Kaheer Suleman, Zichen Zhang, Haoye Lu, and Sumit Pasupalak. World of workflows: a benchmark for bringing world models to enterprise systems. *arXiv preprint arXiv:2601.22130*, 2026.
- [10] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- [11] Hanjiang Hu, Changliu Liu, Na Li, and Yebin Wang. Training task reasoning llm agents for multi-turn task planning via single-turn reinforcement learning. *IEEE Control Systems Letters*, 9:2879–2884, 2025.
- [12] Minki Kang, Jongwon Jeong, Seanie Lee, Jaewoong Cho, and Sung Ju Hwang. Distilling llm agent into small models with retrieval and code tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 106501–106538, 2026.
- [13] Chinmaya Kausik, Adith Swaminathan, and Nathan Kallus. The context gathering decision process: A pomdp framework for agentic search. *arXiv preprint arXiv:2605.07042*, 2026.
- [14] Baohao Liao, Hanze Dong, Christof Monz, Xinxing Xu, Li Dong, and Furu Wei. Multi-turn on-policy distillation with prefix replay. *arXiv preprint arXiv:2607.04763*, 2026.
- [15] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations (ICLR)*, pages 39578–39601, 2024.
- [16] Yuanjie Lyu, Chengyu Wang, Jun Huang, and Tong Xu. From correction to mastery: Reinforced distillation of large language model agents. *arXiv preprint arXiv:2509.14257*, 2025.
- [17] Haoming Meng, Kexin Huang, Shaohang Wei, Chiyu Ma, Shuo Yang, Xue Wang, Guoyin Wang, Bolin Ding, and Jingren Zhou. Sparse but critical: A token-level analysis of distributional shifts in rlvr fine-tuning of llms. *arXiv preprint arXiv:2603.22446*, 2026.
- [18] Seyed Mahed Mousavi, Simone Alghisi, and Giuseppe Riccardi. What does loss optimization actually teach, if anything? knowledge dynamics in continual pre-training of llms. *arXiv preprint arXiv:2601.03858*, 2026.

- [19] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *International Conference on Machine Learning (ICML)*, 2025.
- [20] Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiushi Chen, Dilek Hakkani-Tur, Gokhan Tur, and Heng Ji. Toolrl: Reward is all tool learning needs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 105523–105553, 2026.
- [21] Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Xuanhe Zhou, Yufei Huang, Chaojun Xiao, et al. Tool learning with foundation models. *ACM Computing Surveys*, 57(4):1–40, 2024.
- [22] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations (ICLR)*, pages 9695–9717, 2024.
- [23] Changle Qu, Sunhao Dai, Hengyi Cai, Jun Xu, Shuaiqiang Wang, and Dawei Yin. Matchtir: Fine-grained supervision for tool-integrated reasoning via bipartite matching. *arXiv preprint arXiv:2601.10712*, 2026.
- [24] Qwen Team. Qwen3.6-27B: Flagship-level coding in a 27B dense model, April 2026. URL <https://qwen.ai/blog?id=qwen3.6-27b>.
- [25] Marc’Aurelio Ranzato, Sumit Chopra, Michael Auli, and Wojciech Zaremba. Sequence level training with recurrent neural networks. *arXiv preprint arXiv:1511.06732*, 2015.
- [26] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [27] Guoqing Wang, Sunhao Dai, Guangze Ye, Zeyu Gan, Wei Yao, Yong Deng, Xiaofeng Wu, and Zhenzhe Ying. Information gain-based policy optimization: A simple and effective approach for multi-turn llm agents. *arXiv preprint arXiv:2510.14967*, 2025.
- [28] Shengjie Wang, Guanghe Li, Zonghan Yang, and Yang Gao. Hindsight hint distillation: Scaffolded reasoning for swe agents from cot-free answers. *arXiv preprint arXiv:2605.11556*, 2026.
- [29] Yunan Wang, Minghui Song, Zihan Zhang, Shaohan Huang, Haizhen Huang, Furu Wei, Weiwei Deng, Feng Sun, and Qi Zhang. Group-graph policy optimization for long-horizon agentic reinforcement learning. *arXiv preprint arXiv:2606.22995*, 2026.
- [30] Zhengxian Wu, Kai Shi, Chuanrui Zhang, Zirui Liao, Jun Yang, Ni Yang, Qiuying Peng, Luyuan Zhang, Hangrui Xu, Tianhuang Su, et al. When models judge themselves: Unsupervised self-evolution for multimodal reasoning. *arXiv preprint arXiv:2603.21289*, 2026.
- [31] ZhengXian Wu, Hangrui Xu, Kai Shi, Zhuohong Chen, Yunyao Yu, Chuanrui Zhang, Zirui Liao, Jun Yang, Zhenyu Yang, Haonan Lu, et al. Promsa: Progressive multimodal search agents for knowledge-based visual question answering. *arXiv preprint arXiv:2606.27974*, 2026.
- [32] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *Science China Information Sciences*, 68(2):121101, 2025.
- [33] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- [34] Wanli Yang, Hongyu Zang, Junwei Zhang, Wenjie Shi, Du Su, Jingang Wang, Xueqi Cheng, and Fei Sun. Beyond reasoning: Reinforcement learning unlocks parametric knowledge in llms. *arXiv preprint arXiv:2605.07153*, 2026.
- [35] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- [36] Junjie Ye, Yilong Wu, Songyang Gao, Caishuang Huang, Sixian Li, Guanyu Li, Xiaoran Fan, Qi Zhang, Tao Gui, and Xuanjing Huang. Rotbench: A multi-level benchmark for evaluating the robustness of large language models in tool learning. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 313–333, 2024.

- [37] Junjie Ye, Yilong Wu, Sixian Li, Yuming Yang, Zhiheng Xi, Tao Gui, Qi Zhang, Xuanjing Huang, Peng Wang, Zhongchao Shi, et al. Tl-training: A task-feature-based framework for training large language models in tool use. [arXiv preprint arXiv:2412.15495](#), 2024.
- [38] Junjie Ye, Zhengyin Du, Xuesong Yao, Weijian Lin, Yufei Xu, Zehui Chen, Zaiyuan Wang, Sining Zhu, Zhiheng Xi, Siyu Yuan, et al. Toolhop: A query-driven benchmark for evaluating large language models in multi-hop tool use. In [Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics \(Volume 1: Long Papers\)](#), pages 2995–3021, 2025.
- [39] Junjie Ye, Changhao Jiang, Zhengyin Du, Yufei Xu, Xuesong Yao, Zhiheng Xi, Xiaoran Fan, Qi Zhang, Tao Gui, Xuanjing Huang, et al. Feedback-driven tool-use improvements in large language models via automated build environments. [arXiv preprint arXiv:2508.08791](#), 2025.
- [40] Woongyeng Yeo, Yumin Choi, Taekyung Ki, and Sung Ju Hwang. Hint-sd: Targeted hindsight self-distillation for long-horizon agents. [arXiv preprint arXiv:2605.17873](#), 2026.
- [41] Jiahao Yu, Zelei Cheng, Xian Wu, and Xinyu Xing. Building coding agents via entropy-enhanced multi-turn preference optimization. [arXiv preprint arXiv:2509.12434](#), 2025.
- [42] Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, et al. Glm-5: from vibe coding to agentic engineering. [arXiv preprint arXiv:2602.15763](#), 2026.
- [43] Siliang Zeng, Quan Wei, William Brown, Oana Frunza, Yuriy Nevmyvaka, Yang Katie Zhao, and Mingyi Hong. Reinforcing multi-turn reasoning in llm agents via turn-level credit assignment. In [ICML 2025 Workshop on Computer Use Agents](#), 2025.
- [44] Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. [arXiv preprint arXiv:2601.18734](#), 2026.
- [45] Danna Zheng, Mirella Lapata, and Jeff Z Pan. How reliable are llms as knowledge bases? re-thinking factuality and consistency. [arXiv preprint arXiv:2407.13578](#), 2024.
- [46] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. [arXiv preprint arXiv:2311.07911](#), 2023.